Divide and Conquer

General Method

Divide and Conquer method of designing the algorithm is one of the best known method of solving a problem.

Definition: Divide and Conquer Method is a top-down technique for designing algorithms which consist of dividing the problem into smaller-sub problems hoping that the solutions of the sub problems are easier to find. The solutions of all smaller problems are then combined to get a solution for the original problem.

The Divide and Conquer technique of solving a problem involves three steps at each level of the recursion:

- **Divide:** The Problem is divide into a number of Sub problem.
- **Conquer:** If the sub problems are smaller in size, the problem can be solved using straightforward method. if the sub Problem are larger in size, they are divided into number of sub-Problems of the same type and size. Each of the sub-Problem is solved recursively.
- **Combine:** The Solution of sub Problems are Combined to get the Solution for the larger Problem.

The Divide and Conquer technique is represented as shown Diagram-1

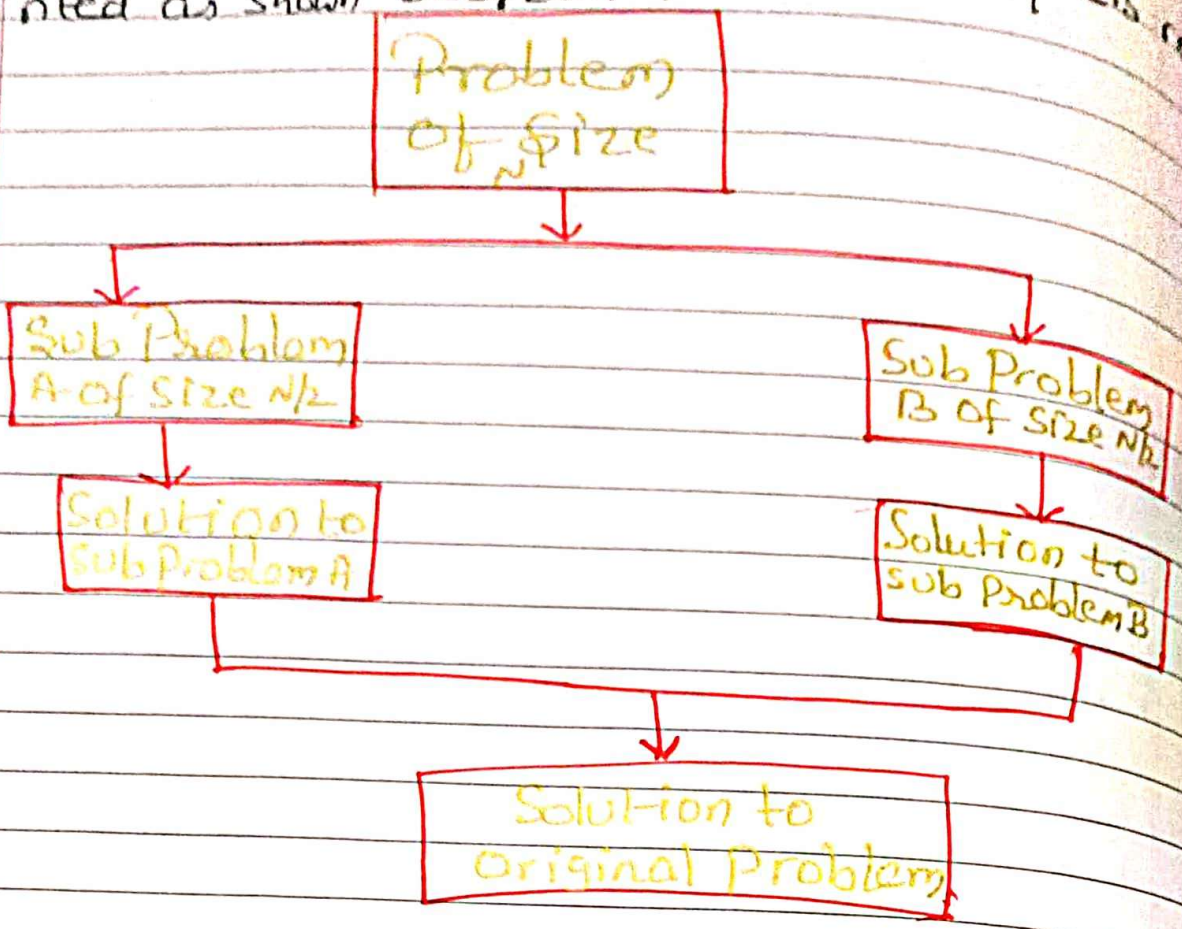


Diagram 1 Divide and Conquer technique

The Control abstraction for divide and Conquer method which shows the flow of Control but precise Meaning is not specified is shown below

Algorithm DANDC(P)

Step1: if small(P) the return S(P)

Step2: Divide p into Smaller(Size) instances $P_1, P_2, P_3 \ldots\ldots P_k$ where $k >= 1$

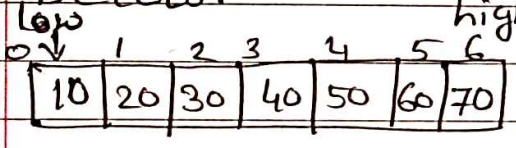
Step 3: Apply DANDC to each of these Problem

A Problem is divide into Smaller instances of the same type as the original Prolem, All these Smaller instances are of the same size. The various problems that can be solved using these approach are shown below.

- Binary Search
- Maximum & Minimum
- Quick Sort
- Merge sort
- Matrix Multiplication using Strassen's algorithm

NOTE: The boundary for a given array is indicated by two index variables low and high. The low index contains the position of 0th element and the index high contains the position of the last element.

EXAMPLE1: An Array Consisting of five elements 10, 20, 30, 40 and 50 can be written as shown below:



NOTE: Whenever low is less than high, it indicates that two or more elements are present in the array.

i.e, if (low < high) more elements are present.

EXAMPLE2: An array Consisting of one element say 10 can be written as shown

Sorting Using Divide and Conquer.

The two searching techniques that closely follows the divide and conquere method are

- Merge Sort
- Quick sort (Partition exchange sort)

Merge Sort

The various steps that are involved while sorting using Merge sort are shown below:

- **Divide:** Divide the given array consisting of n elements into two parts of n/2 elements.
- **Conquer:** Sort the left part of the array and right part of the array recursively using Merge sort.
- **Combine:** Merge the sorted left part and sorted right part of the array to get a single sorted array.

The key operation in merge sort is combining the sorted (array) left part and sorted (array) right part into a single sorted array. This process of Merging of two sorted vectors into a single sorted vector is called Simple Merge. The only necessary condition for this problem is that both arrays should be sorted.

for example, Consider the two sorted vectors, A containing 4 elements and B containg 4 elements. The trace of the above algorithm

Q. Example Sort the following element using Merge Sort == 60, 50, 25, 10, 35, 25, 75, 30

The above element can be sorted using Merge Sort and trace for the same is given below:

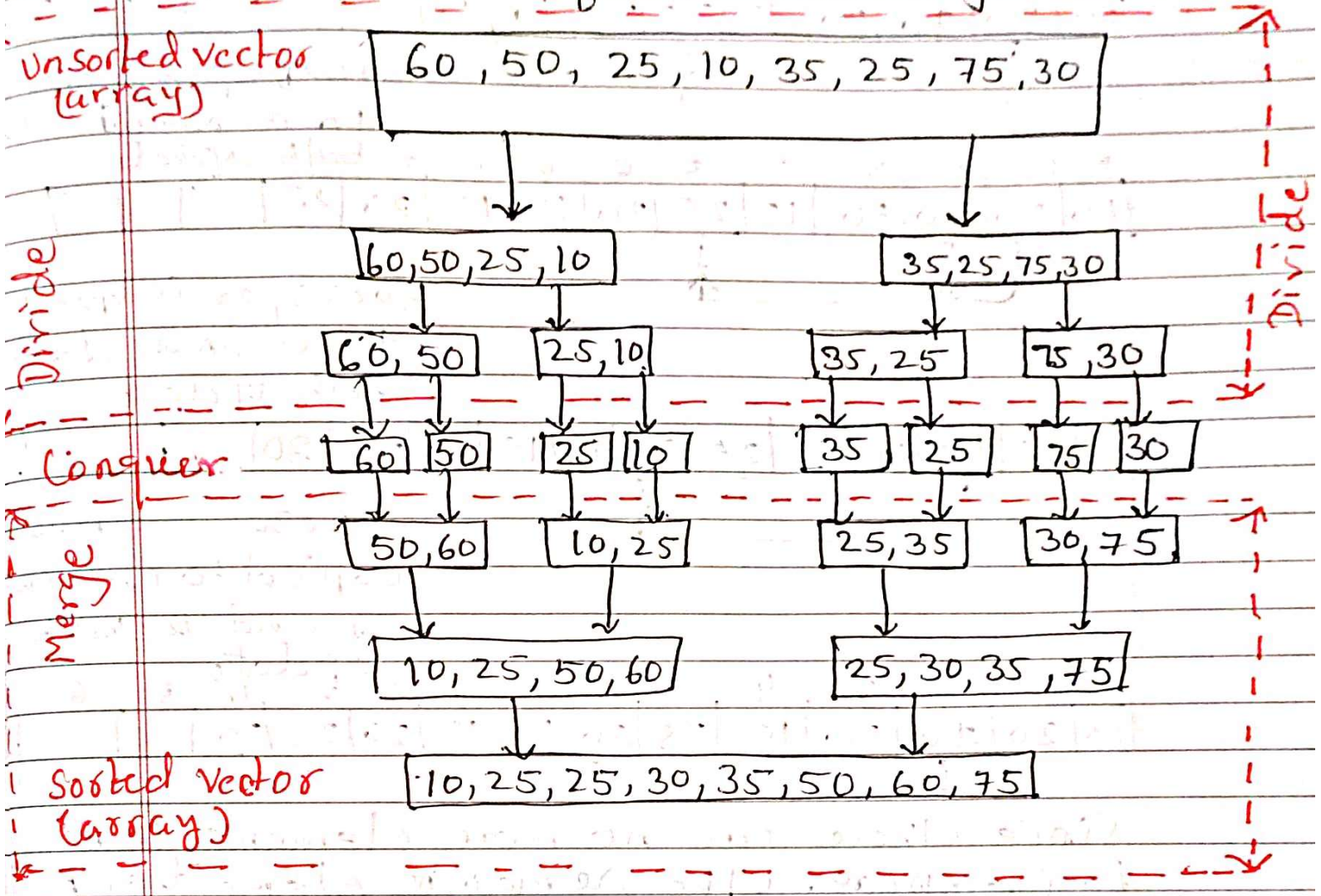


Diagram-1 Trace the Merge sort

ADVANTAGES OF Mergesort Algorithm

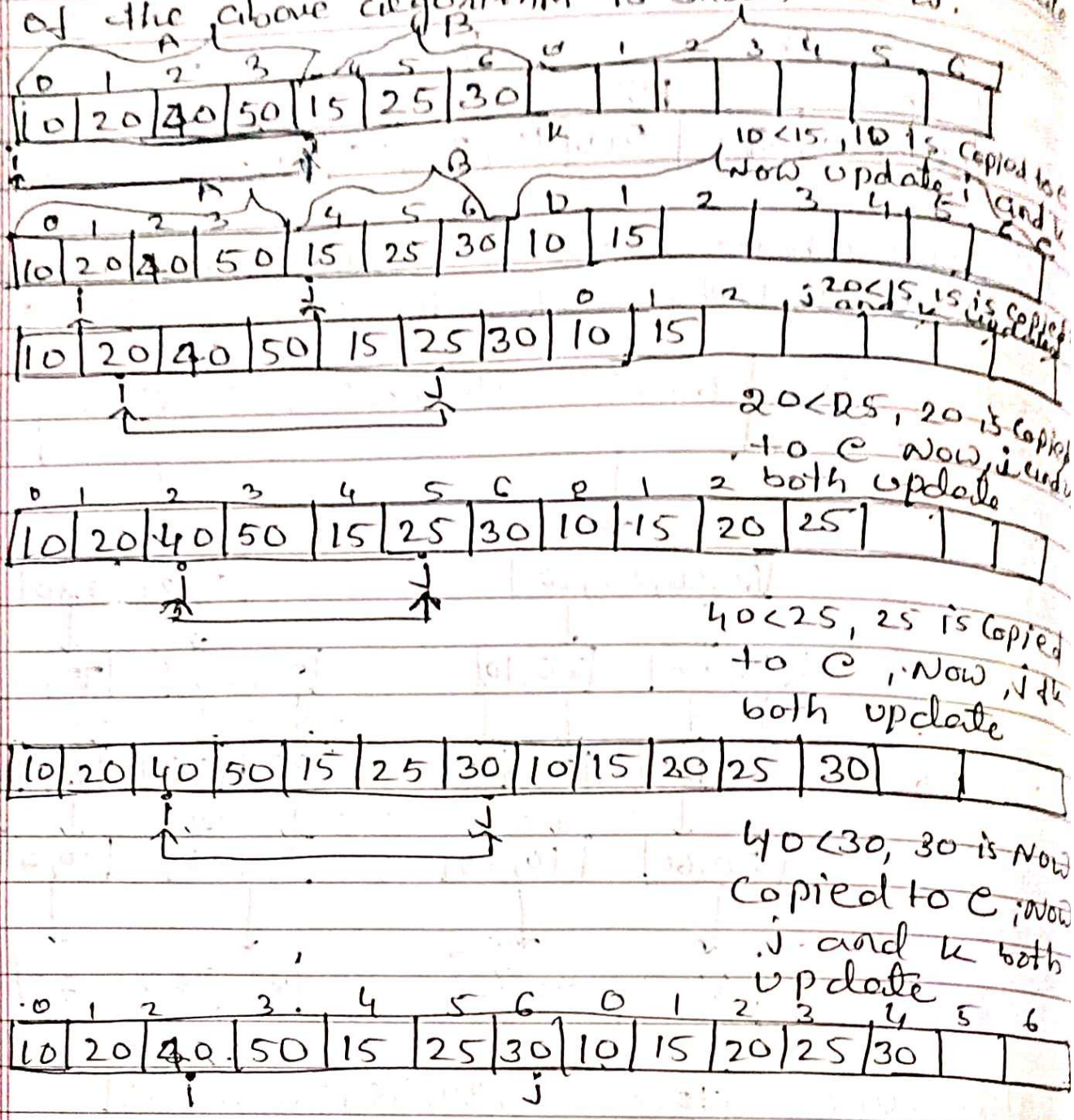
(i) Stable algorithm (ii) Can applied to file any Size. (iii) since I/O is largely sequential tapes can be used iv) Time Complexity is nlogn which

Disadvantages of Merge sort Algorithm

→ It uses More Memory on the stack because of recursion.

or Can be used for [illegible]

-for Example, consider the two sorted vectors, A Containing 4 elements and B Containing 3 elements. The working of the above algorithm is shown below:



Since there are no more element in B to Compare. the remaining elements of A Can be Copied into C, Now 10, 15, 20, 25, 30, 40, 50 are Sorted in the vector C.

← m element → ← n element → ← m+n element →

// Algorithm for SimpleMerge

Algorithm SimpleMerge(a, low, mid, high)

//purpose: Merge two sorted arrays where the first array starts from low to mid and the second mid+1 to high.

//input: A is a sorted from the index position low to mid
// A is sorted from the index position mid+1 to high

//output: A is sorted from index low to high

```
i ← low
j ← mid+1
k ← low
while( i <= mid and j <= high)
        if (A[i] < A[j]) <- // Copy the first part of A to C
            C[k] ← A[i]
            i ← i+1      <- // Point to the index item in the left part of A
            K ← K+1      <- // Points to the next in C.
        else
            C[k] ← A[j] <- // Copy the second part of A to C
            j ← j+1     <- // points to the index item in the right part of the A
            K ← K+1     <- // Points to the next in C.
    end if
end while
        while(i <= mid)
            C[k] ← A[i]  // Copy the remaining
            i ← i+1      // items from left part
            k ← k+1      // of A to C.
```

```
while(j <= high) // copy the remaining items from
                 // right part of A to C
    C[k] ← A[j]
    j ← j+1
    k ← k+1
endwhile

for i = low to high // copy the elements from
    A[i] ← C[i]     // vector C to vector A.
end for
```

Algorithm function for merge sort

```
Algorithm Mergesort(a, low, high)
// purpose: sort the elements of the array between
the lower bound and upper bound.

// input: A is an unsorted vector with low and
high as lower bound and upper bound

// output: A is a sorted vector

if(low > high) return // No elements present in
                      // the list
mid ← (low+high)/2 // Divide the array into two
                   // equal parts.
Mergesort(a, low, mid) // sort the left part of a
Mergesort(a, mid+1, high) // sort the right part of a
SimpleMerge(a, low, mid, high) // Merge the left
                               // part and right
                               // part.
```

Algorithm that Calls Merge sort function to sort the elements.

Step1: [Input The number of elements]
read n

Step2: [Input the array element]
for i ← 0 to n-1 do
read a[i]
endfor

Step3: [Call the Merge Sort function]
Mergesort(a, 0, n-1)

Step4: [output the sorted elements]
for i ← 0 to n-1 do
Print a[i]
end for

Step5: [Terminate the Program]
exit.

Analysis: It is Clear from the algorithm that problem instance is divided into two equal parts. Let us assume that the number of items in the array a are integral multiplies of 2 (i.e $n=2^k$ for $k \geq 0$). The recurrence relation for this algorithm can be Written as:

$$T(n) = \begin{cases} 0 & \text{if } n=1 \text{ (Sorting not required)} \\ T(n/2)+T(n/2)+n & \text{Otherwise.} \end{cases}$$

Time required to sort the left part of the array

Time required to sort the right part of the array

Time required to divide & Merge one instance of a solution with n elements.

NOTE: if $n=1$, there is only one item and hence can be considered as sorted and no time is required for this.

So, $T(1) = 0$ if $n=1$, The above relation can be written as

$$T(n) = \begin{cases} 0 & \text{if } n=1 \\ 2T(n/2) + C_n & \text{Otherwise.} \end{cases} \quad \text{---(i)}$$

Time Complexity using Master Theorem

The general recurrence relation for divide and Conquer Method is

$$T(n) = aT(n/b) + f(n) \quad \text{---(i)}$$

Whereas the recurence relation for Merge Sort is given by

$$T(n) = 2T(n/2) + C_n \quad \text{---(ii)}$$

it is clear from eq^n (2) the above relation that $a=2$, $b=2$ and $f(n) = C_n = C_{n^1} = C_{n^d}$ and so the value of d is one. Substituting these values in the following recurence relation

$$T(n) = \begin{cases} O(n^d) & \text{if } a < b^d \\ O(n^d \log_b n) & \text{if } a = b^d \\ O(n^{\log_b a}) & \text{if } a > b^d \end{cases} \quad \text{---(ii)}$$

we get $a=2$, $b=2$, and $d=1$, $b^d = 2^1 = 2$

it is evident that the second relation $a = b^d$ i.e. $2 = 2^1$ holds good and Time Complexity is given by

$$T(n) = \Theta(n^d \log_b n) = \Theta(n^1 \log_2 n)$$

$$= \Theta(n^1 \log_2 n) = \Theta(n \log_2 n)$$

So, The Time Complexity of an algorithm- Merge Sort is

$$\boxed{T(n) = \Theta(n \log_2 n)}$$

Time Complexity Using Substitute Method

from recurrence relation

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ T(n/2) + T(n/2) + fn & \text{otherwise} \end{cases} \quad \text{--- (1)}$$

We can write eqn (1) as

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ T(n/2) + T(n/2) + n & \text{otherwise.} \end{cases}$$

So

$$T(n) = T(n/2) + T(n/2) + n$$

$$= 2T(n/2) + n \quad \text{--- (ii)}$$

$T(n) = 2T(n/2) + n$ ——— (1)

{ Now Replace n/2 in eq (1)
$T(n/2) = 2T(n/2^2) + \frac{n}{2}$ ——— (2)

Substitute eq (2) in eq (1) we get

$T(n) = 2T(n/2) + n$

$= 2 \cdot \left[2T(n/2^2) + \frac{n}{2}\right] + n$

$= 2^2 T\left(\frac{n}{2^2}\right) + \frac{2n}{2} + n$

$= 2^2 T\left(\frac{n}{2^2}\right) + n + n$

$= 2^2 T\left(\frac{n}{2^2}\right) + 2n$

Similarly $= 2^3 T\left(\frac{n}{2^3}\right) + 3n$

⋮

finally $T(n) = 2^i T\left(\frac{n}{2^i}\right) + in$

$= n \cdot T\left(\frac{n}{n}\right) + in$

= Now Substitute

$= n \cdot T(1) + in$

$= n \cdot 0 + in$

$= 0 + in$

$= in$

$2^i = n$

take log both sides

$\log_2^i = \log_2^n$

$i \log_2^2 = \log_2^n$

So Time Complexity of Merge Sort

$$T(n) = \Theta(n \log_2 n)$$

